

实验二 机器学习开发基础

（一）目的和要求

- （1）掌握 **Numpy 库** 中数组的创建，运算和操作方法
- （2）掌握 **Matplotlib 绘图常用方法**
- （3）了解 **scikit-learn 库** 的基本数据集和主要功能
- （4）掌握 **Tensorflow** 中张量的定义、创建和运算方法

（二）实验知识准备

（1）NumPy

高性能科学计算和数据分析的基础库，它极大地简化了多维数组的操作和处理过程。在实际应用中，大部分数据处理常用库都依赖 NumPy。

（2）Matplotlib 库

Python 的一个 2D 绘图库，借助它可以绘制各种各样的数据图，如折线图、直方图和饼图等。

（3）scikit-learn 库

是一个基于 NumPy、Matplotlib 和 SciPy 的机器学习算法库，它封装了大量的机器学习算法，如有监督学习、无监督学习及半监督学习等，同时内置了大量数据集，其官网首页（<https://scikit-learn.org/stable/>）如图所示。

（4）TensorFlow

一个以张量（Tensor）为基础并对它进行各种运算的深度学习框架。了解张量的定义及其常用运算操作是学习 TensorFlow 的基础。

（5）Keras

是一个使用 Python 编写的开源人工神经网络库，可以作为 TensorFlow 的高阶应用程序接口，进行深度学习模型的设计、调试、评估、应用和可视化等。在 TensorFlow 2.x 中，Keras 成为了 TensorFlow 的官方高级 API，与 TensorFlow 无缝集成，可以直接使用“tf.keras”来调用 Keras 中的函数。

（三）实验内容及步骤

（1）波士顿房价的数据分析及可视化

机器学习的项目中都需要大量的数据，这些数据来源于现实生活、scikit-learn

库或 Keras 库等，然后对数据进行探索是训练模型的必修课。因此，本次实验打算从波士顿房价数据集入手，开始学习这部分内容波士顿房价数据集统计的是 20 世纪 70 年代中期波士顿郊区的房屋数据。该数据集一共有 506 条数据，每条数据包含 13 个特征值和 1 个标签值。特征值包括城镇人均犯罪率、住宅用地所占比例、每栋住宅的平均房间数、到达高速公路的便利指数、城镇非零售业的商业用地所占比例等。标签值是房价的平均值。数据集中各列说明如表 1 所示。

表 1 波士顿房价数据集说明

字段名称	字段类型	字段说明
CRIM	浮点型	城镇人均犯罪率
ZN	浮点型	住宅用地超过 25000 sq.ft. 的比例
INDUS	浮点型	城镇非零售商用土地的比例
CHAS	整型	查理斯河空变量（如果边界是河流，则为 1；否则为 0）
NOX	浮点型	一氧化氮浓度
RM	浮点型	住宅平均房间数
AGE	浮点型	1940 年之前建成的自用房屋比例
DIS	浮点型	到波士顿五个中心区域的加权距离
RAD	整型	辐射性公路的接近指数
TAX	整型	每 10000 美元的全值财产税率
PTRATIO	浮点型	城镇师生比例
B	浮点型	$1000 (Bk - 0.63)^2$ ，其中 Bk 指代城镇中黑人的比例
LSTAT	浮点型	人口中地位低下者的比例
MEDV	浮点型	自住房的平均房价，以千美元计

- 1. 步骤一，导入本项目所需要的模块和包
- 2. 步骤二，使用 load_data()函数从 TensorFlow 的 Keras 库中导入波士顿房价数据集到本地计算机中。参考代码如图 1 所示。

```
1 #参考代码
2 #导入所需要的模块与包
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import tensorflow as tf
6 #导入波士顿房价数据集
7 (x_train, y_train), (_, _) = tf.keras.datasets.boston_housing.load_data(test_split=0)
```

图 1 从 Keras 加载数据集

在 Windows 操作系统中，使用 TensorFlow 的 Keras 库导入的数据集，存放在 C:\Users\User_name\.keras\datasets 文件夹中（User_name 为本地计算机使用 Anaconda 的用户名）。

如果是从本地加载数据集，示例代码如图 2 所示：

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 file_path = r'data/boston_housing.csv' # 替换为你的文件路径
4 boston_data = pd.read_csv(file_path)
5 # 查看数据集的列名和前几行数据
6 print(boston_data.columns)
7 print(boston_data.head())
8 # 假设目标变量是房价，特征变量是其他列
9 X = boston_data.drop('MEDV', axis=1) # 特征
10 y = boston_data['MEDV'] # 目标变量
11 # 划分训练集和测试集
12 x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

图 2 从本地加载数据集

说明：本实验另提供 NPZ 格式的数据集，NPZ 格式文件是 NumPy 库的一种压缩文件格式，用于保存多个 NumPy 数组。它是以 ZIP 格式压缩的，内部可以包含多个 .npy 文件，每个 .npy 文件对应一个 NumPy 数组。

3. 步骤三，查看数据集的特征值 x_train 和标签值 y_train 的类型（均为 NumPy 的 ndarray 对象）。

4. 步骤四，查看数据集的特征值 x_train 和标签值 y_train 的属性，其中，x_train 的维度为 2，y_train 的维度为 1。

x_train 的形状为(506,13)，表示这个数组中共有 506 条数据，每条数据包含 13 个属性，y_train 的形状为(506,)，表示这个数组有 506 条数据，即房价；x_train 和 y_train 元素的数据类型均为 float64。

5. 步骤五 显示数据集的特征值 x_train 和标签值 y_train 的最后 6 条数据。

步骤三至步骤五参考代码如下图 3 所示。

```

1 #查看数据集特征值和标签值的类型
2 print("特征值的类型: ", type(x_train))
3 print("标签值的类型: ", type(y_train))
4 #查看数据集特征值和标签值的属性
5 print("特征值的维度: ", x_train.ndim)
6 print("标签值的维度: ", y_train.ndim)
7 print("特征值的形状: ", x_train.shape)
8 print("标签值的形状: ", y_train.shape)
9 print("特征值元素的类型: ", x_train.dtype)
10 print("标签值元素的类型: ", y_train.dtype)
11 #显示数据集中部分数据值
12 print("特征值的最后6行数据: ")
13 print(x_train[-6:])
14 print("标签值的最后6个数据: ")
15 print(y_train[-6:])

```

图 3 访问数据集

6. 步骤六，可视化数据并设置图形中的中文字体为“黑体”；初始化 titles 列表，其值为 x 轴的标签；创建画布并设置画布的大小。
7. 步骤七，使用循环语句绘制各张子图，循环体完成以下功能。
 - a. 绘制子图，子图类型为散点图，x 轴为各个特征值，y 轴为标签值（房价）
 - b. 设置 x 轴和 y 轴的标题
 - c. 设置子图的标题。
8. 步骤五 使用 `tight_layout()` 函数自动调整子图参数，使之填充整个图像区域后，设置整张图的标题，并使用 `show()` 函数显示图形。

步骤六至步骤七参考代码如图 3 所示。

```

1 #参考代码
2 #设置图形中的中文字体为“黑体”
3 plt.rcParams['font.sans-serif']=['SimHei']
4 titles=["CRIM", "ZN", "INDUS", "CHAS", "NOX", "RM", "AGE", "DIS",
5 "RAD", "TAX", "PTRATIO", "B", "LSTAT", "MEDV"]
6 plt.figure(figsize=(16,16)) #创建画布并设置画布的大小
7 #绘制子图
8 for i in range(13):
9     plt.subplot(4,4,(i+1))
10    plt.scatter(x_train[:,i], y_train)
11    plt.xlabel(titles[i])
12    plt.ylabel("Price($1000's)")
13    plt.title(str(i+1)+ ". " +titles[i]+"-Price")
14 #自动调整子图，使之填充整个图像区域
15 plt.tight_layout()
16 plt.show()

```

图 3 可视化数据集

实验结果如下图 4 所示：

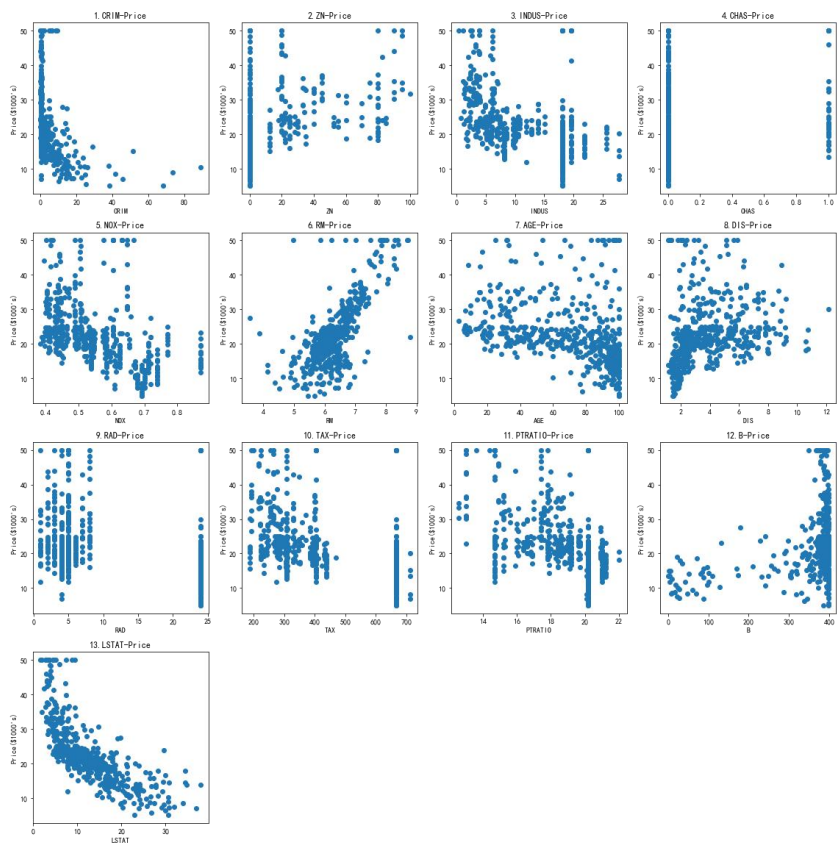


图 4 数据集可视化结果

从图 4 的散点图可以看出，有些特征值与目标变量之间存在着比较明显的关系，如每栋住宅的平均房间数（子图 6）、低收入人口的比例（子图 13）等，而其他一些特征值对目标变量没有明显的影响，如城镇非零售业的商业用地所占比例（子图 3）、是否有河流穿过（子图 4）等。

（2）实训项目

导入 scikit-learn 库中的鸢尾花数据集，显示数据集特征值和标签值的类型、属性和部分数据，并绘制花瓣长度和花瓣宽度与鸢尾花品种的散点图。鸢尾花数据集各列说明如表 2 所示。

表 2 鸢尾花数据集说明

字段名称	字段类型	字段说明
SepalLength	浮点型	萼片长度（厘米）
SepalWidth	浮点型	萼片宽度（厘米）
PetalLength	浮点型	花瓣长度（厘米）
PetalWidth	浮点型	花瓣宽度（厘米）
Class	字符型	类别：Setosa, Versicolour, Virginica

鸢尾花数据集中共有 150 条数据，分为 3 类，每类 50 条数据，每条数据包含 4 个特征值和 1 个标签值，其中特征值分别是花萼长度、花萼宽度、花瓣长度和花瓣宽度，标签值表示鸢尾花的种类，分别是 Setosa、Versicolor 和 Virginica，用整数 0、1 和 2 表示

1. 步骤一，使用 `load_iris()` 函数导入 `scikit-learn` 库中的鸢尾花数据集。
2. 步骤二，获取特征值和标签值，使用 `features` 来存储特征值，使用 `labels` 来存储标签值。
3. 步骤三，访问数据集数据。完成以下任务。
 - a. 查看数据集的特征值 `features` 和标签值 `labels` 的类型。
 - b. 查看数据集的特征值 `features` 和标签值 `labels` 的属性，包括维度、形状、元素的数据类型和元素总个数。
4. 步骤四，显示数据集的特征值 `features` 和标签值 `labels` 的前 10 条数据。绘制花瓣长度和花瓣宽度与鸢尾花品种的散点图。要求在绘制散点图时，使用不同颜色的点来区分不同品种的鸢尾花。实验效果如图 5 所示。
 - a. 花瓣长度的特征值存储在 `feature[:,2]` 中，花瓣宽度存储在 `feature[:,3]` 中。
 - b. 函数 `scatter(x,y,c=None,cmap=None)` 中，将参数 `c` 指定为一个列表或数组，所绘制图形的颜色，可以随这个列表或数组中元素的值而变换，变换所对应的颜色由参数 `cmap` 所提供。

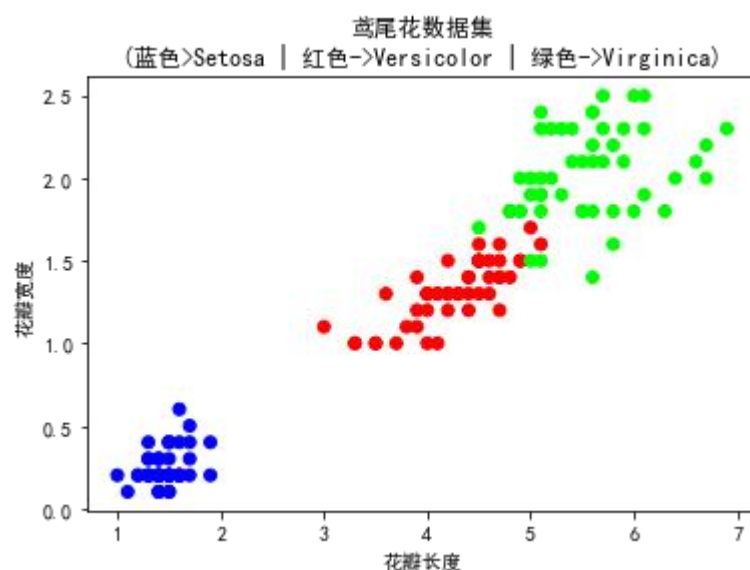


图 5 鸢尾花数据集散点图